Arithmetic Coding

The Huffman method is simple, efficient, and produces the best codes for individual data symbols. However, it doesn't always produce the most effective codewords for a given set.

In fact, the only case for which Huffman produces ideal VLCs (codes whose average size equals the entropy) is when the symbols occur at probabilities that are negative powers of 2 (i.e., numbers such as 1/2, 1/4, or 1/8). This is because the Huffman method assigns a codeword with an integer number of bits to each symbol in the given symbol set.

Information theory shows that a symbol with probability 0.4 should ideally be assigned a 1.32-bit code, because $-\log_2(0.4) \approx 1.32$. The Huffman method, however, assigns such a symbol a code of 1 or 2 bits.

Sadly, as long as the number of bits we assign to a codeword are integer-based, there will always be a bloated difference between the number of encoded bits and the actual bits required by entropy. So, to fix that, we must move away from assigning integer-based codewords to symbols at a 1:1 ratio.

This is where *arithmetic coding* comes in. Rather than assigning codewords to symbols in a 1:1 fashion, this algorithm transforms the entire input stream from a set of symbols to one (excessively long) numeric value, whose $\log_2$ representation is closer to the true value of entropy for the stream.

The magic of arithmetic compression is a transform that it applies to the source data in order to create a single output number, which takes fewer bits to represent than the source data itself.

Story Time: Where Arithmetic Coding Originated

Peter Elias first proposed the concept behind arithmetic compression in the early 1960s. But, it wasn't until a decade later that Jorma Rissanen from IBM published some of the first suitable research for its implementation—alongside a massive patent.

As a result, for the next couple of decades, arithmetic compression pretty much fell off the map, due to an impossibly aggressive patent strategy enforced by IBM. The patent problem was so enormous and arithmetic compression so good, a different algorithm called *Range Coding* was invented in 1979. It basically did the same thing as arithmetic compression but was free from patents.

In the early 2000s, the patent finally expired. Arithmetic compression took off again and achieved status as the gold standard for the current generation of statistical encoders.

In fact, the majority of modern compression formats for archives (such as LZMA and BZIP) and audio and video (such as JPEG, WebP, webM, and H.264) all work with an arithmetic compression system as their statistical encoding step.

Finding the Right Number

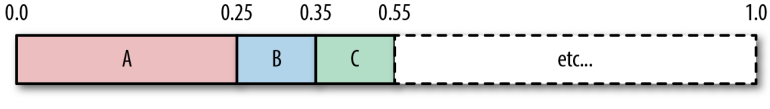
Arithmetic coding works by transforming a single string into a number that requires fewer bits to represent than the original string. For example, "TOBEORNOT" could be represented by 236712[6], whose ceil($-\log_2(236712)$) = 18 bits. Compare this with the ASCII version of "TOBEORNOT," which sits at 56 bits.

But this isn't as easy as just picking a number at random, as we did earlier. Arithmetic coding goes through a complex process of calculating this number from an input stream. The trick is that choosing the number is actually a modification on the binary search algorithm that we introduced in Chapter 2 (you know, the chapter you were not supposed to skip).

If you recall from that chapter, you can use a binary search to output 0/1 bits to catalog the search's no/yes decision process as we were checking whether a number fit into one of two spaces against a pivot value. But what if we had four spaces? Each decision would then output 2 bits (for one-fourth of the number range, respectively). Still makes pretty good sense, right?

Arithmetic coding kinda works along this path, but with some serious modifications.

Arithmetic coding creates a number space [0,1)[7] and subdivides it based on the probability of symbols in the data stream. So, "A" would be given [0,0.25) if its probability were 25%, and B, with a probability of 10%, would then be given [0.25,0.35), and so on, as shown here:

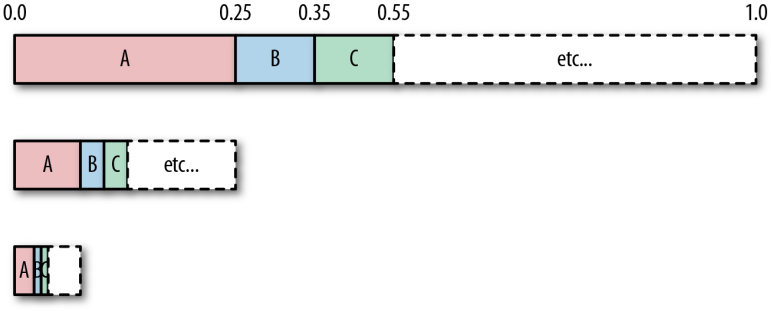


When the encoder reads a symbol, it finds the range for that symbol. For example, if A were read, the range [0.0,0.25) would be used. After a symbol is read, the encoder will subdivide that symbol range and assign new range values for symbols proportionally.

6 It actually isn't. This is just an example number to show you the general steps.

7 The [0,1) notation indicates that the number 0 is included in the range, and 1 is not, making the range 0 through 0.99999…, with as many 9s as you need.

For example, if our stream provided three A symbols in a row, the encoder would subdivide the A range three times, as illustrated here:



Basically, each symbol recursively divides its range until we reach the end of the input stream. After that's done, we have a final range of values, such as [0.253212, 0.25421). The number you output, is a value in this range. So for our example, for an input string of AAA, the output could be anything in the range [0,0.015625).

Encoding

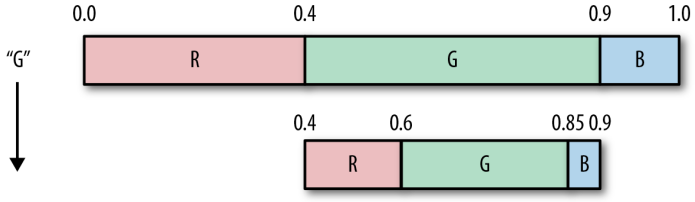
Let's encode a complete example with three symbols, R, G, and B, with respective probabilities of 0.4, 0.5, and 0.1. We assign them ranges in the interval [0,1) according to their probability, as shown in the following table (note that the entropy of this table is 1.36):

Symbol	Probability	Interval
R	0.4	[0,0.4)
G	0.5	[0.4,0.9)
B	0.1	[0.9,1)

The following diagram shows the same information represented in number-line form of the ranges for three symbols, R, G, and B:

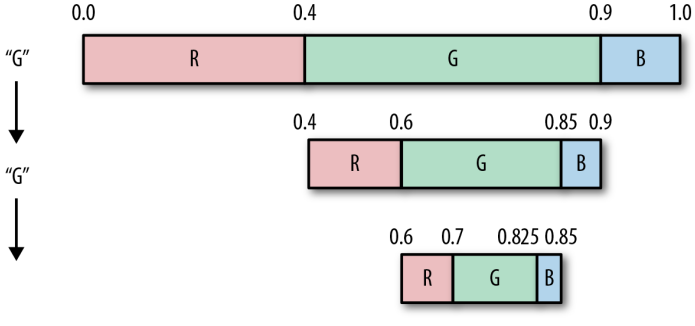
Using this probability setup, let's encode the string "GGB."

The first symbol we read from the input stream is "G." Because the interval of G, according to our table, is [0,4,0.9), we subdivide the number space between the values of 0.4 and 0.9 according to probabilities, which gives us a new range of intervals, as shown in the updated figure and table that follow.



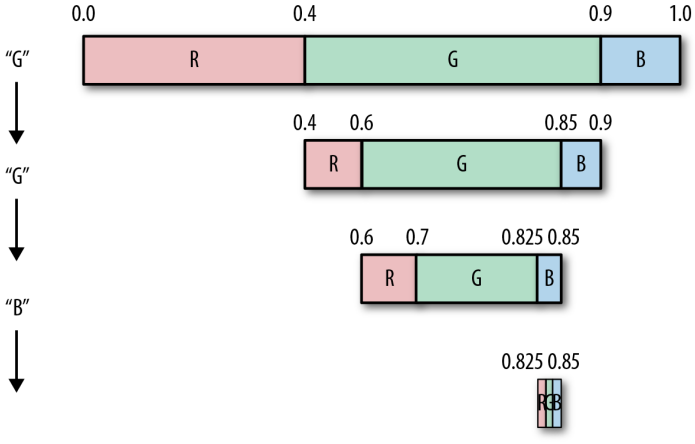
Symbol	Probability	Updated interval
R	0.4	[0.4,0.6)
G	0.5	[0.6,0.85)
B	0.1	[0.85,0.9)

We now read the second symbol from the input stream. It's another G, so we again subdivide the G range of numbers, which is between 0.6 and 0.85, and update the intervals as shown in the following figure and the table.



Symbol	Probability	Updated interval
R	0.4	[0.6,0.7)
G	0.5	[0.7,0.825)
B	0.1	[0.825,0.85)

And finally, we read and subdivide again for the letter B, and update the figure and table once more.



Symbol	Probability	Updated interval
R	0.4	[0.825,0.835)
G	0.5	[0.835,0.8475)
B	0.1	[0.8475,0.85)

Congratulations! You just arithmetically encoded a stream!

Picking the Right Output Value

So, we've subdivided the range intervals, but what number do we output as the final result?

Our final interval for B is [0.825,0.85), and any number from that range will serve the purpose of letting us reconstruct the original string, giving us some choices between 0.825 and 0.849999.... This being a book about compression, we want to use the number that we can represent with the fewest bits (and as close to our 1.36 bits entropy goal) as possible.

The number in this range that requires the fewest bit is 0.83.

Finally, because the decoder "knows" that this is a floating-point number, we can drop the leading 0 for additional savings and end up with 83, which with $\log_2(83) = 7$ bits, gives us 1.42 bits per symbol, which is almost at entropy.

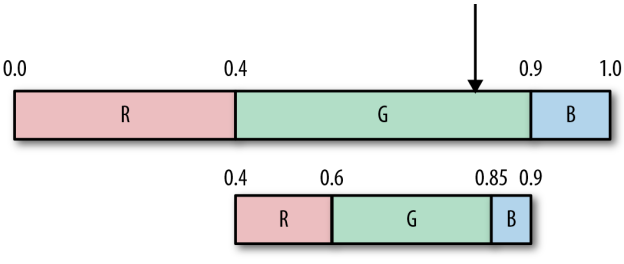
Decoding

Decoding from this final number is pretty straightforward—basically the encoding process in reverse. Just as with encoding, we begin by creating the segmented range between [0,1) based on the probabilities, as demonstrated here:

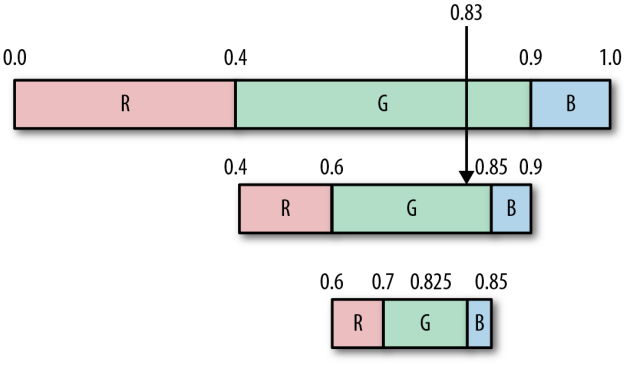
We then take our input value, 83, add the leading "0" to get 0.83, and find which interval it falls into. We output the symbol associated with that interval.

In our case, 0.83 falls in the range 0.4,0.9, giving us the value G to output.

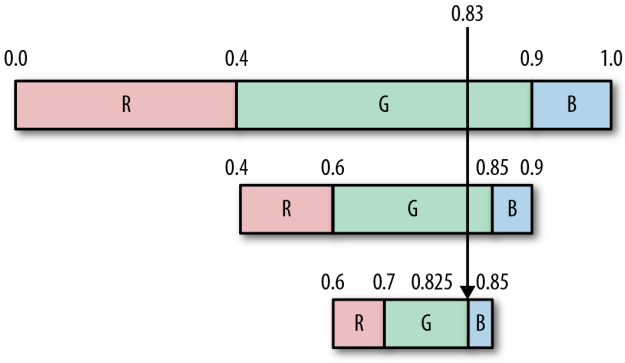
We then subdivide the G space according to the probabilities (same as when we were encoding).



To get our second symbol, we repeat the process. Our input value is still 0.83, which falls (again) in G's range, so we output another G, and subdivide again, as shown here:



Continuing one more time, 0.83 falls in the range 0.825,0.85, which is the range for the symbol B, so we output B.



This gives us our final decoded stream as GGB.

Pretty nifty, eh? Our decoding process basically works by drawing a line through each recursive space and outputting whatever symbol we land in at that time.